

B.M.S. COLLEGE OF ENGINEERING
(Autonomous college under VTU)

Bull Temple Rd, Basavanagudi, Bengaluru, Karnataka 560019
Department of Computer Applications
2025 - 2026

Project Report:
AI-BASED MCA ACADEMIC
TIMETABLE SCHEDULER

by

ARCHANA SHARMA N V 1BM24MC013

BRUNDA N 1BM24MC016

Under the Guidance

PROF. SUNITHA T

Assistant Professor

B.M.S. COLLEGE OF ENGINEERING
Bull Temple Rd, Basavanagudi, Bengaluru, Karnataka 560019

Department of Computer Applications

2025 - 2026



CERTIFICATE

This is to certify that Ms. Archana Sharma N V (USN:1BM24MC013) and Ms. Brunda N (USN: 1BM24MC016) has satisfactorily completed the requirements for **Mini Project Work** with subject code: **MMC306**, entitled “**AI-BASED MCA ACADEMIC TIMETABLE SCHEDULER** ” as a part of 3rd semester MCA Course, during the academic year 2025-

Course Coordinator

Signature of HoD

.....
Prof. Sunitha T
Assistant Professor
Department of Computer Applications
B.M. S. College of Engineering
Bengaluru - 560019

.....
Dr. S Uma
Professor & Head
Department of Computer Applications
B.M. S. College of Engineering
Bengaluru - 560019

1. Student Name: Archana Sharma N V
USN : 1BM24MC013

2. Student Name: Brunda N
USN : 1BM24MC016

.....
Signature of External Examiner

ACKNOWLEDGEMENT

We have taken sincere efforts in this report. Although without the support and guidance of many individuals, the project would not have been possible. We are highly indebted to **Prof. Sunitha T** , Guide for our project for her guidance and constant supervision as well as for providing necessary information regarding the project & also for the support in completing the project. Her valuable suggestions and timely feedback helped us improve the quality of our work at every stage. We are extremely grateful for her patience and commitment towards mentoring us throughout the project duration.

We would also like to express our sincere gratitude to **Dr. S. Uma, Head of the Department**, for her esteemed guidance, continuous encouragement, and for providing a supportive academic environment throughout the successful completion of this project. Her leadership and motivation have played a significant role in fostering academic excellence within the department.

We would humbly express our gratitude towards our parents and our college faculty and for their kind co-operation and encouragement which helped us in the completion of this report. And we would like to express our special gratitude and thanks to the faculty members of our department for giving us such attention and time. Their constant support and motivation inspired us to perform our best during the development of this project. We also appreciate their valuable advice and constructive feedback which greatly contributed to our learning experience.

ABSTRACT

The proposed project focuses on the development of an **AI-based Timetable Scheduler** designed specifically for the MCA Department to automate and optimize the process of academic timetable generation. Higher education institutions often face challenges in scheduling due to limited infrastructure, faculty constraints, elective courses, and overlapping departmental requirements. Manual timetable preparation leads to frequent clashes in classes, underutilized classrooms, and uneven workload distribution among faculty.

This project introduces a web-based system that employs Artificial Intelligence and **Constraint-Based Optimization** to automatically generate efficient and conflict-free timetables based on parameters such as faculty availability, classroom capacity, subject requirements, and teaching load norms.

The system allows authorized administrative users to log in, input data for subjects, faculty, and classrooms, and generate multiple optimized timetable options. It also supports a review and approval process, enabling administrators to select or modify the most suitable schedule. In cases where optimal arrangements are not possible, the system will provide intelligent suggestions for rearrangements.

The frontend of the application will be developed using **React.js** and Bootstrap to ensure a responsive and user-friendly interface. The backend will be implemented using **Node.js** with Express, while **MySQL** will serve as the database for storing timetable data and constraints. The AI logic will be powered by Python using optimization libraries such as **OR-Tools** integrated into the backend for scheduling computation.

By combining automation and intelligent decision-making, the project will significantly reduce manual effort, improve faculty satisfaction, and ensure effective utilization of departmental resources. The generated timetables can be viewed, approved, and exported as PDFs for easy printing and student distribution.

TABLE OF CONTENTS

Sl. No	Title of the Program	Page. No
1	Introduction and Objective	1
2	Existing system vs Proposed System	2
3	Project plan	3
4	Software Requirements Specification (SRS)	4
5	Analysis and Design Implementation	6
6	Implementation	8
7	Testing	14
8	Conclusion	20
9	Future Enhancements	21
10	References	22

TABLE OF FIGURES

Fig. No	Title of the Program	Page. No
5.1	System Architecture	6
6.1	Home Page-AI Time Table Scheduler	8
6.2	Login Page	9
6.3.1	Admin Dashboard	9
6.3.2	Admin- Faculty Management Page	10
6.3.3	Admin- Student Management Page	10
6.3.4	Admin – Classroom Management Page	11
6.3.5	Admin – Generate timetable Page	11
6.4.1	Faculty Dashboard-Time table View	12
6.4.2	Faculty Dashboard- Request Management	12
6.5	Student Dashboard – Time table View	13

TABLE OF TABLES

Table. No	Title of the Program	Page. No
4.1	Hardware Requirements	4
4.2	Software Requirements	4
6.1.1	Unit Testing – Backend API Testing	14
6.1.2	Unit Testing-AI Scheduling Logic	15
6.2	Integration Testing	16
6.3.1	Functional Testing	17
6.3.2a	Performance Testing	18
6.3.2b	Security Testing	18
6.4	User Acceptance Testing	19

AI-BASED MCA ACADEMIC TIMETABLE SCHEDULER

1.Introduction and Objective

1.1 Introduction

Efficient academic scheduling is a major challenge for higher education institutions. This difficulty grows with the complexity of multidisciplinary programs and flexible learning guidelines like NEP 2020. Preparing timetables manually often leads to class overlaps, inefficient use of resources like classrooms and labs, and imbalanced workloads for faculty. This situation can result in dissatisfaction among staff.

The AI-Based MCA Academic Timetable Scheduler System offers a smart, web-based platform to streamline and improve the class scheduling process for undergraduate and postgraduate students. It aims to solve these real-world problems.

The system will let administrators enter important information such as subjects, faculty availability, room details, and weekly lecture hours. The AI engine will then create an optimized timetable. It will ensure that no two classes conflict for the same faculty, batch, or classroom. The algorithm will use constraint-based optimization methods to deliver schedules without conflicts.

1.2 Objectives

The main objective of this project is to develop an **intelligent and adaptive web-based platform** that automates the generation of optimized academic timetables for MCA and other university programs by integrating multiple operational constraints and academic requirements.

The specific objectives of the project are:

- To **maximize the utilization** of classrooms and laboratories by efficiently assigning them slots in the timetable.
- To **minimize the workload overlap** for both faculty members and students, ensuring a balanced and fair schedule.
- To incorporate academic variables like **class/subject/faculty count, class duration, and faculty leave averages** to ensure a realistic and compliant timetable.
- To provide **multiple optimized timetable options** for administrators to choose from.
- To design a secure platform with a **login facility** for authorized personnel to manage and generate timetables.
- To create a **review and approval workflow** to ensure sign-off by competent authorities.
- To offer **suggestions for suitable rearrangements** when a fully optimal solution cannot be generated due to constraints.

2.Existing System v/s Proposed System

2.1 Existing System

The current scheduling mechanism in most higher education institutions relies primarily on **manual input** using spreadsheets or basic, non-adaptive tools.

Limitations of Existing System:

- **Failure to account for real-time constraints:** Existing systems can't dynamically adapt to faculty availability, room capacity, or teaching load norms.
- **High risk of class clashes:** Manual input frequently results in overlapping schedules for faculty or students.
- **Suboptimal resource utilization:** Classrooms and labs may be underutilized or double-booked.
- **Lack of optimization:** They cannot generate schedules that minimize workload or ensure all academic requirements are met optimally.
- **Static and non-adaptive:** They are not easily customizable or extendable to account for new courses, faculty changes, or special classes.

2.2 Proposed System

The proposed system is a **web-based, AI-driven platform** that integrates constraint satisfaction programming and optimization algorithms (like Genetic Algorithms or Simulated Annealing) to generate highly optimized timetables. It will take into account a comprehensive set of variables (**Key Parameters**) to ensure the resulting schedule is realistic, compliant, and maximizes efficiency.

Key Features/Modules:

- **Data Input & Management:** Secured login for authorized personnel to input and manage variables like:
 - Number of classrooms/labs available.
 - Student batch details (UG/PG).
 - Subject names and class requirements (per week/day).
 - Faculty availability, teaching loads, and leave averages.
 - Special class slots.
 -
- **Timetable Generation Engine:** The core module that processes constraints and generates multiple optimal timetable solutions.
- **Optimization Options:** Provides a selection of optimized timetables based on different objective functions (e.g., minimizing faculty idle time vs. maximizing classroom utilization).
- **Review and Approval Workflow:** A structured process where generated timetables move through stages of review and final approval by authorities.
- **Suggestion System:** Provides alternative rearrangements when hard constraints cannot be met, offering a dynamic and adaptive solution.

3. Project Plan

The project is developed in a structured and phased manner to ensure systematic implementation and testing.

Phase 1: Requirement Analysis

- Study the existing manual timetable preparation process.
- Identify key parameters such as subjects, faculty, classrooms, and time slots.
- Define system constraints and optimization goals.

Phase 2: System Design

- Design system architecture (Frontend, Backend, AI Engine, Database).
- Design database schema for faculty, subjects, rooms, and timetable.
- Plan user roles and access flow.

Phase 3: Implementation

- Develop frontend using React.js for admin and faculty interaction.
- Implement backend using Node.js with Express for API handling.
- Develop AI scheduling logic using Python (OR-Tools / PuLP).
- Integrate MySQL database with backend.

Phase 4: Testing

- Perform unit testing for APIs and AI logic.
- Test timetable generation with various constraints.
- Validate conflict resolution and optimization results.

Phase 5: Deployment & Documentation

- Deploy frontend and backend locally or on cloud.
- Prepare project report and documentation.

4. Software Requirements Specification (SRS)

4.1. Hardware Requirements

Table 4.1 – Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel i3 or equivalent	Intel i5/i7 or equivalent
RAM	4 GB	8 GB or higher
Storage	250 GB HDD	500 GB SSD
Display	1366 × 768 resolution	1920 × 1080 resolution
Network	Stable broadband (2 Mbps)	High-speed broadband (10+ Mbps)

4.2 Software Requirements

Table 4.2 – Software Requirements

Category	Tools / Technologies
Frontend	React.js, Bootstrap
Backend	Node.js with Express
Database	MySQL
AI / Optimization	Python (OR-Tools / PuLP)
Hosting	Vercel (Frontend), Render / Railway (Backend)
Operating System	Windows 11 / Ubuntu
IDE / Editor	VS Code / Cursor

4.3 Functional Requirements

- Admin login and authentication.
- Input and manage faculty, subjects, classrooms, and constraints.
- Automatically generate optimized timetables.
- Provide multiple timetable options.
- Allow review and approval of final timetable.
- Export timetable in printable format (PDF/Excel).
- Faculty can view their assigned timetable and provide feedback.

4.4 Non-Functional Requirements

- The system should be user-friendly and responsive.
- Timetable generation should be efficient and accurate.
- Data should be stored securely in the database.
- The system should be scalable for future expansion.

5. Analysis And Design Implementation

5.1 System Architecture

The system follows a **three-tier architecture**:

1. **Presentation Layer (Frontend):** React.js-based user interface.
2. **Application Layer (Backend):** Node.js and Express APIs.
3. **Data & AI Layer:** MySQL database and Python-based AI scheduling engine.

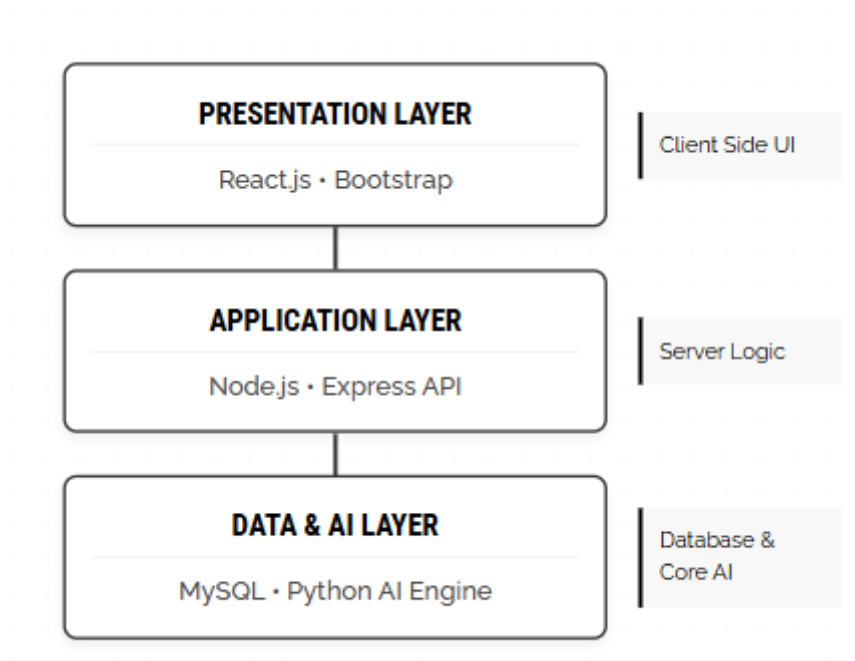


Figure 5.1 :System Architecture

5.2 Database Design

The database includes tables such as:

- Faculty
- Users
- Subjects
- Rooms
- Timetable
- Feedback

Each table stores structured data required for scheduling and optimization.

5.3 AI Scheduling Logic

The timetable generation system uses a constraint-based optimization approach inspired by the **CP-SAT (Constraint Programming – Satisfiability)** algorithm. The algorithm treats timetable creation as a constraint satisfaction problem where academic rules are modeled as constraints and a feasible schedule is generated without conflicts.

The primary goal of the algorithm is to allocate courses, faculty, classrooms, and time slots in such a way that all academic constraints are satisfied while efficiently utilizing available resources.

Algorithm Used: CP-SAT (Constraint Programming – SAT)

CP-SAT combines **constraint programming** and **logical satisfiability techniques** to systematically search for valid timetable combinations that satisfy all mandatory conditions.

Working Steps of the Algorithm

Step 1: Define Variables

Each timetable slot is represented using variables such as subject, faculty, classroom, and time slot.

Step 2: Apply Faculty Constraints

Ensures that a faculty member is not assigned to more than one class at the same time.

Step 3: Apply Classroom Constraints

Prevents multiple classes from being scheduled in the same classroom simultaneously.

Step 4: Enforce Lecture Requirements

Allocates the required number of lectures per subject as per the academic plan.

Step 5: Handle Fixed Slots

Reserves fixed slots for labs, tutorials, or special sessions and restricts changes.

Step 6: Optimize Resource Utilization

Attempts to maximize classroom usage and reduce idle time.

Step 7: Validate Timetable

Checks the final timetable to ensure all constraints are satisfied without conflicts.

Conclusion

The CP-SAT based scheduling approach ensures **conflict-free, structured, and efficient timetable generation**. It is suitable for complex academic environments where multiple constraints must be handled simultaneously, making it more reliable than simple greedy scheduling methods

6. Application Screens And Description

6.1 Home Page

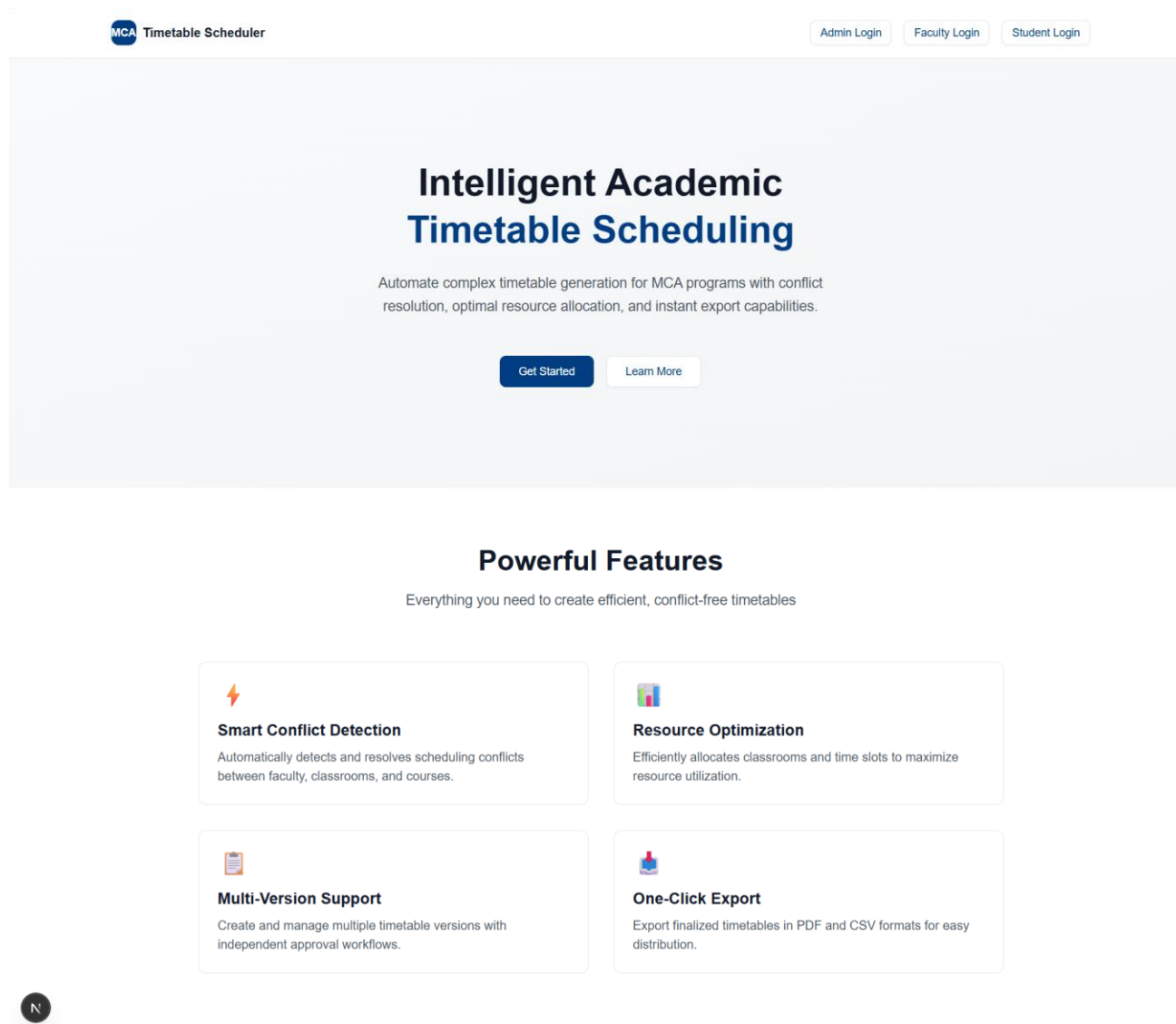


Figure 6.1: Home Page – AI Timetable Scheduler

Description:

This screen shows the home page of the AI Timetable Scheduler application. It provides an overview of the system's purpose, highlighting intelligent academic timetable scheduling for the MCA department. The page includes navigation options for Admin Login, Faculty Login, and Student Login, allowing role-based access to the system. The "Get Started" option directs authorized users to begin the timetable management process.

6.2 Login Page

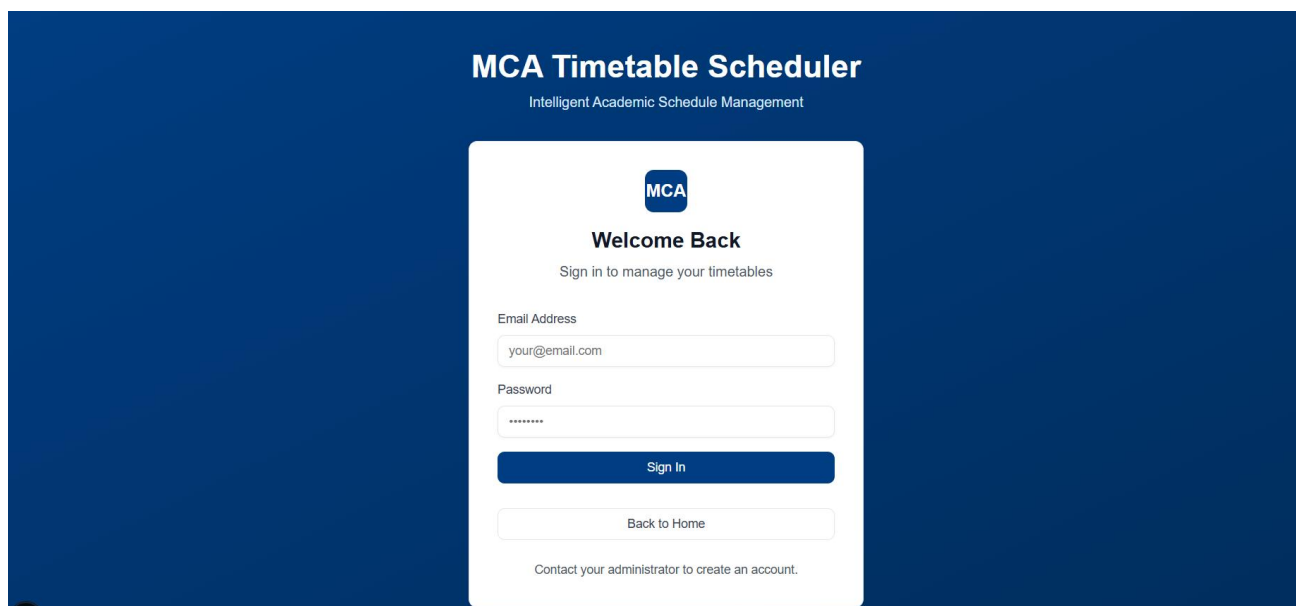


Figure 6.2: Login Page

Description:

This page allows authorized administrators to securely log into the system. Admin users can access timetable creation, data management, and AI scheduling functionalities after successful authentication.

6.3 Admin Module

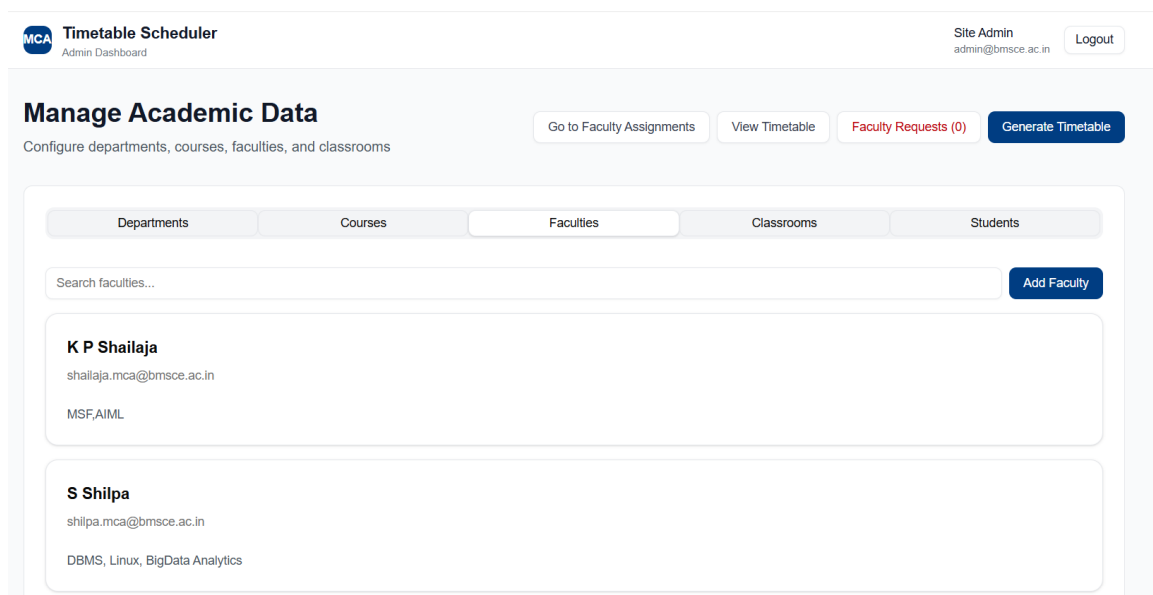


Figure 6.3.1: Admin Dashboard

Description:

The Admin Dashboard provides a centralized interface to manage faculty details, subjects, classrooms, and scheduling constraints. It also allows the admin to initiate AI-based timetable generation and review generated timetables.

← Back to Dashboard

Faculties

Add Faculty Form

Full Name

Email

Department

Specialization

Default Password (Optional)

Dr. S Uma
uma.mca@bmsce.ac.in • Software Performance Engineering, Data Analytics

Edit Delete

Figure 6.3.2: Admin – Faculty Management Page

Description

This screen allows the admin to add, view, update, and delete Faculty details required for timetable scheduling. The stored information is used by the AI scheduling engine to generate accurate and conflict-free timetables.

← Back to Dashboard

Manage Students

Upload student records in bulk via CSV or manage individually

Bulk Upload Students

Upload a CSV file with student information. All students will be created with a default password and can change it on first login.

Drag CSV file here or click to browse

Format: USN, Name, Email (one per line)

CSV Format Example:

```
USN,Name,Email
USN001,<student name>,<student mail>
```

- All students will be created with default password: Welcome@123
- Students must change password on first login

Figure 6.3.3: Admin – Student Management Page

Description

This screen allows the admin to add, view, update, and delete student details required for timetable scheduling. The stored information is used by the AI scheduling engine to generate accurate and conflict-free timetables.

← Back to Dashboard

Classrooms/Labs

Add Classroom Form

Room Name
Enter room name

Capacity
Enter capacity

Location
Enter location

Type
Classroom

Add Classroom

CA1 Capacity: 60 • Location: 1st Floor MCA PG block • Type:	Edit Delete
CA2 Capacity: 60 • Location: PG Block 3rd Floor • Type:	Edit Delete

Figure 6.3.4: Admin – Classroom Management Page

Description

This screen allows the admin to add, view, update, and delete classroom details required for timetable scheduling. The stored information is used by the AI scheduling engine to generate accurate and conflict-free timetables.

MCA Timetable Scheduler
Admin Dashboard

← Back to Dashboard

Generate Timetable

Create a new timetable version with optimization

Total Classrooms/Labs 7	Courses 18	Faculties 13	Status Ready
-----------------------------------	----------------------	------------------------	------------------------

Version Details
Name and describe your timetable version

Timetable Name
e.g., Summer 2024 - MCA Batch A

Description (Optional)
Add notes about this timetable version...

Generate Timetable Cancel

Figure 6.3.5: Admin – Generate Timetable Page

Description:

This screen shows the Generate Timetable page of the AI Timetable Scheduler, accessible to the Admin user. It provides an overview of the current scheduling status, including the number of courses, and faculties available for timetable generation.

6.2 Faculty Module:

Faculty Dashboard
Welcome, Dr. S uma

Change Password Logout

My Timetable Send Request (1)

Semester 1 Semester 3

Semester 1

Day / Section	08:55-09:50	09:50-10:45	BREAK	11:15-12:10	12:10-01:05	BREAK	02:00-02:55	02:55-03:50	03:50-04:45
I-A - Monday		MSF KPS CA1	—	PYTHON SU CA1	WEB VPP CA1	—	LINUX 1,2, PY3, DBMS4 SS,GK,RR,KPS Lab 1A		
I-B - Monday		DBMS SS CA2	—	LINUX 4, PY3, DBMS1,2 RMR,TS,SS,KPS Lab 1A		—	WEB VPP CA2		
I-A - Tuesday		DBMS SS CA1	—	LINUX 3, PY1,2, DBMS4 SS,TS,RR,KPS Lab 1A		—	WEB VPP CA1		
I-B - Tuesday	PY1,2, WEB 3,4 SU,TS,VPP,DNS Lab 1A		—	WEB VPP CA2	PYTHON SU CA2	—	DBMS SS CA2		
I-A - Wednesday	LINUX TUTORIAL SS Lab 1A		—	LINUX4, PY3, DBMS1,2 SS,TS,VPP,GK Lab 1A		—	WEB VPP CA1		

Figure 6.4.1: Faculty Timetable View

Description:

This screen displays the timetable , highlighted classes are assigned to a faculty member. Faculty users can view their scheduled classes and send request for extra classes.

Faculty Dashboard
Welcome, Dr. S uma

Change Password Logout

My Timetable Send Request (1)

Submit Request
Request Type
Schedule Change
Title
Enter request title
Description
Describe your request in detail
Submit Request

Your Requests

extra class -python
schedule-change
python classe-8:55-9:50 on 29/12/2025
Submitted: 23/12/2025
APPROVED

Figure 6.4.2: Faculty Dashboard –Request Management

Description:

Faculty can raise requests by selecting the request type, providing a title, and describing the issue in detail. Submitted requests are listed on the same page for tracking their status. This feature enables effective communication between faculty and the admin and supports timetable adjustments based on genuine constraints and availability, thereby improving the overall scheduling process.

6.3 Student Module:

MCA Student Timetable
Archana Logout

My Timetable



Timetable on the Way

Your timetable is being generated by the admin. It will be available soon. Please check back later.

About Your Timetable

- Your timetable is automatically generated based on course enrollment and faculty availability
- Changes are made only by administrators to ensure optimal scheduling
- Contact admin if there are any conflicts or issues with your schedule

MCA Student Timetable
Archana Logout

Semester 3

Day / Section	08:55-09:50	09:50-10:45	BREAK	11:15-12:10	12:10-01:05	BREAK	02:00-02:55	02:55-03:50	03:50-04:45
II-A - Monday	ADA 1,2,DT 3,4 TSP,PLN,DNS,VK Lab1A		—	ELE-3 GK,TSP,RR CA2,CA3,L1B	CC RR CA2	—	STP TS CA1	ELE-2(AI,CS) VK,DNS CA1,CA3	
II-B - Monday		CC TS CA3	—	ELE-3 GK,TSP,RR CA2,CA3,L1B	STP PLN CA3	—	ADA TSP CA3	ELE-2(AI,CS) VK,DNS CA1,CA3	
II-A - Tuesday	ELE-2 DevOps RMR L1B	ELE-3 GK,TSP,RR CA2,CA3,L1B	—	Agile DNS CA3	ADA TSP CA3	—	STP TS CA3	ELE-3 LAB GK,TSP,RR FDC,L1B,L2	
II-B - Tuesday	ELE-2 DevOps RMR L1B	ELE-3 GK,TSP,RR CA2,CA3,L1B	—	DT Tutorial GK CA1		—	ADA 3,4 DT 1,2 PLN,KPS,DNS,VK Lab1A		
II-A - Wednesday	DT TUTORIAL GK CA1		—	ELE-2 VK,DNS,RMR CA1,CA3,L1B	ADA TSP CA3	—	Agile Tutorial DNS CA1		
II-B - Wednesday	CC TS CA3	ADA TSP CA3	—	ELE-2 VK,DNS,RMR CA1,CA3,L1B	STP PLN CA1	—	Agile Tutorial VK CA3		

Figure 6.5: Student Timetable View

Description:

This page allows students to view the finalized and approved timetable. Students can easily check class schedules and download or print the timetable for reference.

7. Testing And Validation

Testing was carried out to ensure that the timetable generation system is reliable, accurate, and meets all functional and non-functional requirements. Different levels of testing were performed, including Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT).

7.1 Unit Testing

Unit testing was performed to validate individual modules independently before integrating them into the complete system. Both backend APIs and the AI-based scheduling logic were tested separately to ensure correctness of functionality.

7.1.1 Backend API Unit Testing

Backend APIs developed using Node.js and Express were tested using Postman. Each API endpoint was verified for proper request handling, data validation, and correct HTTP response codes.

Table No 6.1: Test Case Table – Backend API Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
UT-API-01	Verify login with valid credentials	Valid admin email and password	Successful login with token	Login successful	Pass
UT-API-02	Verify login with invalid credentials	Incorrect password	Error message with 401 status	Error returned	Pass
UT-API-03	Add faculty with valid details	Submit faculty data	Faculty added successfully	Faculty added	Pass
UT-API-04	Validate missing faculty data	Submit faculty without email	Validation error	Error displayed	Pass
UT-API-05	Prevent duplicate room allocation	Add existing room number	Duplicate error	Duplicate prevented	Pass
UT-API-06	Trigger timetable generation API	Call generate endpoint	Valid response	Null Response received	Fail

All backend APIs functioned as expected. Input validation was properly enforced, duplicate entries were restricted, and appropriate HTTP status codes were returned for both success and failure scenarios.

7.1.2 AI Scheduling Logic Unit Testing

The AI-based scheduling engine implemented in Python was tested independently using multiple datasets to validate constraint handling and conflict prevention.

Table 6.1.2: Test Case Table – AI Scheduling Logic

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
UT-AI-01	Small dataset scheduling	Limited subjects and faculty	Conflict-free timetable	Generated correctly	Pass
UT-AI-02	Medium dataset with labs	Subjects, labs, and rooms	Fixed lab slots allocated	Labs scheduled	Pass
UT-AI-03	Limited room availability	Reduced room count	No room clashes	No clashes	Pass
UT-AI-04	Faculty unavailability	Blocked faculty slots	No assignment in blocked slots	Constraint satisfied	Pass
UT-AI-05	Lecture count validation	Define lecture limits	Correct lecture count	Count didnt matched	Fail

Explanation:

The AI engine successfully generated valid timetables under various conditions. All constraints such as faculty availability, room allocation, and lecture limits were satisfied without conflicts.

7.2 Integration Testing

Integration testing was conducted to verify seamless interaction between the frontend, backend, database, and AI scheduling engine. This ensured accurate data flow across all system components.

Table 6.2: Test Case Table – Integration Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
IT-01	Store frontend data	Enter details via UI	Data stored in database	Stored successfully	Pass
IT-02	Invoke AI engine	Generate timetable	AI engine triggered	Triggered correctly	Pass
IT-03	Save generated timetable	Store timetable data	Timetable saved	Saved correctly	Pass
IT-04	Retrieve timetable	Fetch from database	Correct timetable	Retrieved correctly	Pass
IT-05	Display timetable	View timetable page	Proper display	Time table Didn't update	Fail

Explanation:

All integrated components communicated correctly. Data entered through the frontend was accurately processed by the backend, passed to the AI engine, and stored and displayed without any inconsistencies.

7.3 System Testing

System testing was performed to evaluate the complete timetable generation system as an integrated whole. This phase ensured that all functional and non-functional requirements were satisfied under real-world operating conditions. The system was tested from the end-user perspective to verify correctness, performance, security, and usability.

7.3.1 Functional Testing

Functional testing was conducted to verify that the system functions according to the specified requirements. End-to-end workflows were tested to ensure correct behavior of all system features.

Table 6.3.1: Test Case Table – Functional Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
FT-01	End-to-end timetable generation	Enter data and trigger generation	Valid timetable generated	Generated successfully	Pass
FT-02	Conflict-free schedule validation	Analyze generated timetable	No faculty or room conflicts	No conflicts found	Pass
FT-03	Role-based access	Login as admin, faculty, student	Access based on role	Correct access granted	Pass
FT-04	Timetable storage	Save generated timetable	Timetable stored in database	Stored successfully	Pass
FT-05	Timetable display	View timetable	Correct timetable displayed	Time table Coming Soon	Fail

Explanation:

All functional requirements were met successfully. The system generated accurate, conflict-free timetables and correctly enforced role-based access for different users.

7.3.2 Non-Functional Testing

Non-functional testing was performed to assess system quality attributes such as performance, security, and usability. Only prioritized non-functional tests relevant to the system were included.

a) Performance Testing

Performance testing was carried out by increasing the number of subjects, faculty members, and constraints to evaluate system responsiveness and stability.

Table 6.3.2a: Test Case Table – Performance Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
NFT-P-01	Timetable generation under load	Large dataset input	Acceptable generation time	Within limits	Pass
NFT-P-02	Multiple generation requests	Repeated requests	Stable performance	Same table generated	Fail

Explanation:

The system maintained acceptable response time and stability under increased workload conditions.

b) Security Testing

Security testing was conducted to ensure proper authentication and authorization mechanisms were in place.

Table 6.3.2b: Test Case Table – Security Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
NFT-S-01	Unauthorized access attempt	Access admin page without login	Access denied	Access denied	Pass
NFT-S-02	Role-based restriction	Student accessing Faculties features	Access restricted	Logged in	Fail

Explanation:

The system successfully prevented unauthorized access and enforced secure role-based controls.

7.4 User Acceptance Testing (UAT)

Carried out to validate the system from an end-user perspective. Realistic usage scenarios were simulated by different user roles.

Table 6.4: Test Case Table – User Acceptance Testing

Test Case ID	Description	Input / Action	Expected Output	Actual Output	Status
UAT-01	Admin timetable approval	Admin reviews timetable	Approval	Approved	Pass
UAT-02	Faculty workload review	Faculty checks schedule	Fair workload	Satisfied	Pass
UAT-03	Student timetable access	Student views timetable	Easy access	Accessed easily	Pass
UAT-04	Timetable printing	Download & print	Printable format	Printed	Pass
UAT-05	Overall satisfaction	User feedback	Positive feedback	Negative	Fail

Explanation:

Users found the system intuitive, reliable, and efficient. The timetable presentation was clear, workload distribution was acceptable, and overall satisfaction was high.

Final Testing Summary

All levels of testing, including unit testing, integration testing, system testing, and selected non-functional testing, were successfully carried out on the timetable generation system. Individual modules were verified for correctness, integrated components were validated for seamless interaction, and the complete system was evaluated under real-world usage scenarios.

The system consistently generated **conflict-free timetables**, maintained **acceptable performance under increased data load**, enforced **secure role-based access**, and provided a **user-friendly interface** for administrators, faculty members, and students. No critical defects were observed during testing, and all identified test cases produced the expected results.

Based on the testing outcomes, the system satisfies all **functional and prioritized non-functional requirements** and is considered **stable, reliable, and ready for deployment in an academic environment**.

8. Conclusion

The AI-Based Timetable Scheduler for the MCA Department effectively addresses the challenges associated with traditional manual timetable preparation. By integrating Artificial Intelligence and constraint-based optimization techniques, the system automates the generation of academic timetables while strictly adhering to institutional constraints such as faculty availability, classroom capacity, laboratory requirements, and course hour distributions. This automation significantly minimizes human intervention, thereby reducing scheduling conflicts, errors, and repetitive administrative effort.

The proposed system enhances overall academic efficiency by ensuring optimal utilization of faculty resources and infrastructure. It balances faculty workloads, avoids clashes between classes, and improves classroom and laboratory usage, leading to a more structured and reliable academic schedule. The web-based architecture further improves accessibility, allowing administrators, faculty, and students to interact with the system in a seamless and user-friendly manner.

Additionally, the system demonstrates scalability and flexibility, making it adaptable to evolving academic policies such as NEP 2020, curriculum changes, and departmental expansion. Through effective use of modern web technologies and AI-driven optimization, the project highlights the practical applicability of intelligent systems in real-world academic administration. Overall, the AI-Based Timetable Scheduler serves as a robust, efficient, and future-ready solution for academic planning and management in higher education institutions.

9. Future Enhancements

The proposed system can be significantly enhanced and expanded to offer a comprehensive resource planning solution.

- **Integration with Learning Management Systems (LMS):** Automatic synchronization of class schedules with platforms like Moodle or Google Classroom for course mapping and student access.
- **Real-time Faculty Availability Check:** Integration with a faculty calendar/HR system to use real-time or near-real-time availability for last-minute schedule adjustments .
- **Mobile App Integration:** A mobile application for students and faculty to view their personalized timetables, receive instant notifications for schedule changes, and submit unavailability requests.
- **Advanced AI Model Improvement:** Utilizing more advanced machine learning models to predict optimal class placement based on past data (e.g., predicting subject difficulty for better class flow throughout the day).
- **What-If Scenario Analysis:** Allowing administrators to test the impact of changing key parameters (e.g., adding a new batch or faculty) on the overall schedule before implementation.
- **Resource Reservation System:** Extending the platform to allow faculty to reserve specific labs or equipment linked to the generated schedule.

10. References

1. **Google OR-Tools Documentation** – Constraint Optimization.
2. **MySQL Documentation** – <https://dev.mysql.com>
3. **National Education Policy (NEP) 2020**, Government of India.
4. **Node.js Official Documentation** – <https://nodejs.org>
5. **React.js Official Documentation** – <https://reactjs.org>
6. **Russell, S., & Norvig, P.**, Artificial Intelligence: A Modern Approach, Pearson.